



Name: Cohort/Batch: Date: Score:

APPDYNAMICS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Observability

TOTAL: 10 QUESTIONS

Q1 What is AppDynamics and what problem in Observability does it solve?

Q6 How does AppDynamics handle reliability and high availability?

Q2 What are the core components or concepts in AppDynamics?

Q7 What observability does AppDynamics provide out of the box?

Q3 How does AppDynamics compare to its main alternatives?

Q8 What are common performance bottlenecks in AppDynamics?

Q4 How do you get started with AppDynamics for a new project?

Q9 What security best practices apply when running AppDynamics?

Q5 What configuration options most affect AppDynamics's behavior in production?

Q10 What mistakes do teams commonly make when adopting AppDynamics?



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 AppDynamics is a tool or platform that

Mitigation

AppDynamics is a tool or platform that automates or simplifies a specific concern in the Observability domain, reducing manual effort and operational complexity for engineering teams.

A2 AppDynamics has key building blocks that work

Primitives

AppDynamics has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A3 AppDynamics trades off simplicity against feature richness

Enhancing

AppDynamics trades off simplicity against feature richness differently than its alternatives. Choose AppDynamics when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A4 Install AppDynamics via its package manager or

Gotchas

Install AppDynamics via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A5 Production-critical settings typically include concurrency

Configuration

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A6 AppDynamics provides HA through leader election, Observability

AppDynamics provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A7 AppDynamics exposes metrics (Prometheus endpoint or Information

AppDynamics exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A8 Performance bottlenecks typically stem from misconfigured Performance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A9 Run AppDynamics with the principle of least

Assessment

Run AppDynamics with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A10 Common pitfalls include skipping the documentation and Organization

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.